

Country-Pack Regulatory Isolation Target Architecture

Project: Stoquify

Decision date: 26 July 2026

Selected architecture: Regulatory Isolation Boundary

Decision: Proceed with controlled country-pack isolation for development and integration.

Executive decision

Adopt a single **Regulatory Isolation Boundary** built from three coordinated patterns:

1. A ports-and-adapters boundary around country-pack resolution.
2. Separate integration and production-release verification profiles.
3. Transactional-outbox processing for deferred fiscalization.

This is the cleanest and least disruptive path. It removes country-pack readiness from ordinary feature development without deleting regulatory contracts, weakening financial controls, or creating a dangerous global bypass flag.

The design reuses mechanisms Stoquify already has: the published country-pack resolver, separate development and production statutory gates, `BusinessEvent` and `BusinessEventOutbox`, idempotent fiscal-document creation, compliance submission workers, explicit payroll blocked states, and AP review-required metadata.

A new microservice would add deployment and operational friction. Scattered conditional checks would create long-term coupling. A global disable flag could leak into production. The Regulatory Isolation Boundary avoids all three.

Current coupling

The direct country-pack resolver is imported by five active runtime consumers:

- `services/compliance/country-pack-hooks.ts`
- `services/payroll/compensation.service.ts`
- `services/payroll/payroll-control.service.ts`
- `services/payroll/payroll-setup-readiness.service.ts`
- `services/purchasing/ap-control.service.ts`

POS has a separate direct dependency on fiscal-document creation in `services/pos/pos.service.ts`.

Fiscalization currently runs inside the sale transaction, so a country-pack or fiscal-document exception can roll back an otherwise valid sale.

The CI coupling begins in `package.json`, where ordinary repository verification inherits the production country-pack gate. Pull-request CI therefore treats normal feature integration partly like a production-promotion ceremony.

Regulatory capability boundary

Create a stable application-facing contract:

```
RegulatoryExecutionMode =  
  SANDBOX  
  DEFERRED  
  PRODUCTION  
  
RegulatoryDecision<T> =  
  AUTHORITATIVE(value, provenance)
```

```
NON_AUTHORITATIVE(value, simulationSource, SIMULATION_ONLY)
PENDING_COUNTRY_PACK(reasonCode, requestedParameter)
```

The application boundary should expose:

```
RegulatoryCapabilityPort.resolve<T>(  
  parameter,  
  context  
) -> RegulatoryDecision<T>
```

Mandatory rules:

- PENDING_COUNTRY_PACK never contains a fabricated value.
- NON_AUTHORITATIVE values come only from versioned fixtures.
- Simulated results cannot be converted into authoritative results.
- Provenance remains mandatory for authoritative results.
- Feature modules import the port, never country-packs/resolve.ts.

New boundary files

- services/regulatory/ports/regulatory-capability.port.ts
- services/regulatory/regulatory-capability.service.ts
- services/regulatory/adapters/published-country-pack.adapter.ts
- services/regulatory/adapters/deferred-regulatory.adapter.ts
- services/regulatory/runtime/regulatory-runtime-class.ts
- services/regulatory/__tests__/regulatory-capability-boundary.test.ts

The existing country-pack resolver remains the implementation behind the published-country-pack adapter.

Runtime selection and production guardrails

The regulatory mode must be derived from a validated deployment classification. It must not be independently selectable through a bypass flag.

Deployment class	Derived regulatory mode
Local development	DEFERRED
Automated tests	SANDBOX
Preview environment	DEFERRED
Dedicated sandbox	SANDBOX
Production	PRODUCTION
Unknown production-style runtime	PRODUCTION

Required guardrails:

- Unknown environments default to PRODUCTION.
- Production platform metadata overrides local configuration.
- COUNTRY_PACK_DISABLED and similar variables are prohibited by a static gate.
- Production startup rejects deferred and sandbox adapters.
- Production fiscal-document services retain existing adapter and expert-review checks.

- Release verification confirms that no non-production adapter is registered for production.

CI and verification split

Keep the existing `policy: gates` command as the backward-compatible production policy chain.

Add:

```
policy:gates:integration
verify:repo -> policy:gates:integration
verify:ci -> verify:repo
verify:release -> verify:repo + policy:gates + release gates
```

The integration profile must retain:

- Tenant and API boundaries.
- Ledger-close truth.
- Payment and cash truth.
- Purchasing and AP controls.
- Inventory boundaries.
- Offline POS replay.
- Payroll immutability.
- Migration safety.
- Hard-delete prevention.
- Regulatory hard-code detection.
- Error boundaries.
- Development country-pack readiness.

The integration profile must exclude:

- Expert approval.
- Production artifact approval.
- Live authority-adapter certification.
- Operational promotion evidence.
- Independent release approval.

Files affected:

- `package.json`
- `.github/workflows/ci.yml`
- `scripts/statutory-country-pack-development-gate.js`
- `scripts/ci-release-readiness-gate.js`
- `scripts/release-evidence-ratchet.js`
- Corresponding tests under `scripts/__tests__`

Pull requests continue running `verify:ci`, but the command becomes integration-oriented. `verify:release` remains the only production-promotion path.

Deferred POS fiscalization

Remove immediate fiscal-document creation from the POS sale transaction.

The transaction should persist:

- The completed sale.
- Payment effects.
- Inventory effects.
- Balanced ledger postings.
- Audit records.
- The `pos.sale.finalized` business event.
- One `pos.fiscalization.requested` outbox message.

Recommended idempotency key:

```
pos-sale:{saleId}:fiscalization:v1
```

The outbox payload should contain immutable identifiers and hashes:

```
schemaVersion
organizationId
salesOrderId
postingBatchId
locationId
terminalId
sourcePayloadHash
requestedAt
```

The worker must rehydrate the completed sale and posted ledger batch. It must not trust mutable client payloads.

Additive schema changes

Extend the existing Prisma enums:

```
BusinessOutboxChannel += FISCALIZATION
BusinessOutboxStatus += DEFERRED
```

DEFERRED represents intentional pending work rather than a technical failure and does not consume retry attempts.

No new queue table is required. The existing `BusinessEventOutbox` already supplies transactional persistence, idempotency, attempt limits, lock ownership, scheduling, failure states, and dead-letter handling.

No existing fiscal-document columns should be removed or weakened.

New worker files

- `services/compliance/fiscalization-request.schemas.ts`
- `services/compliance/fiscalization-worker.service.ts`
- `services/compliance/fiscalization-reconciliation.service.ts`
- `services/compliance/__tests__/fiscalization-worker.service.test.ts`
- `services/compliance/__tests__/fiscalization-reconciliation.service.test.ts`

Regulatory result	Worker action
AUTHORITATIVE	Create the fiscal document idempotently
NON_AUTHORITATIVE	Create a sandbox-only document when permitted
PENDING_COUNTRY_PACK	Mark the outbox request DEFERRED
Retryable technical failure	Reschedule with backoff
Invalid or conflicting source	Dead-letter with audit evidence

POS and receipt contracts

Replace the immediate fiscal-document response with:

```
fiscalization:
requestId
status:
DEFERRED
PENDING_COUNTRY_PACK
SANDBOX_QUEUED
FISCALIZED
authority:
NON_AUTHORITATIVE
AUTHORITATIVE
```

Affected files:

- services/pos/pos.service.ts
- services/pos/receipt.service.ts
- components/pos/ProfessionalPOSSystem.tsx
- services/pos/offline-sync.service.ts
- Corresponding POS, receipt, and offline-sync tests.

Operational receipts may still be produced in development, but must display:

- NON_FISCAL_DEVELOPMENT
- PENDING_COUNTRY_PACK
- NOT A CERTIFIED FISCAL DOCUMENT

In production, a pending country-pack result must block any claim that the receipt is legally certified. The commercial transaction and its audit evidence remain intact.

Payroll, AP, and accounting behavior

Payroll

Payroll previews may use existing versioned fixtures through a separate simulation provider. They remain non-payable, non-declarable, non-postable as authoritative tax, and excluded from certified close.

Existing BLOCKED_REQUIRES_EXPERT_REVIEW behavior remains mandatory for authoritative calculation.

Accounts payable

AP already catches regulatory-resolution failures and records UNRESOLVED, operatorActionRequired, and the error code and message. Migrate its resolver import to the new port while preserving this behavior.

Accounting close

Draft close views may include non-authoritative previews. Certified close must reject:

- NON_AUTHORITATIVE
- PENDING_COUNTRY_PACK
- Deferred statutory obligations.
- Unreconciled fiscalization requests.

Reconciliation when country packs return

When an approved country pack is published:

1. Select deferred outbox requests by country and effective date.
2. Re-resolve each request through the authoritative adapter.
3. Verify the original sale, posting batch, and source hash.
4. Create the fiscal document using the existing idempotency key.
5. Record pack version, resolution hash, and legal reference.
6. Mark the outbox request SENT.
7. Emit `pos.fiscalization.completed`.
8. Produce a report containing processed, deferred, conflicting, and failed counts.

Historical events must not be rewritten. New reconciliation events supersede their pending status.

Implementation sequence

Phase 1: CI split

Change verification commands, CI expectations, and gate tests. This immediately removes expert approval from ordinary feature integration.

Phase 2: Regulatory boundary

Introduce the port and migrate the five direct resolver consumers. Add an import-boundary gate prohibiting direct country-pack imports outside the adapter.

Phase 3: Additive outbox migration

Add FISCALIZATION and DEFERRED. Deploy the migration before changing POS behavior.

Phase 4: POS outbox production

Write the fiscalization request in the sale transaction and remove immediate fiscal-document creation.

Phase 5: Worker and receipt states

Enable sandbox processing, pending-state UI, retries, reconciliation, and operational visibility.

Phase 6: Production restoration

After approval, enable the published adapter through the production release process and reconcile deferred obligations.

Test strategy

Required automated tests:

- Integration CI passes while the production country-pack gate remains blocked.
- Release verification continues to fail on missing expert approval.
- No unauthorized direct imports of country-pack implementations remain.
- POS completes when country-pack resolution is unavailable.
- A sale cannot commit without its fiscalization outbox message.
- Concurrent workers cannot create duplicate fiscal documents.
- Replaying the same request returns the same fiscal document.
- Source-hash conflicts are dead-lettered.
- Deferred requests do not consume retry attempts.

- Sandbox receipts are visibly non-authoritative.
- Production receipts cannot claim certification while deferred.
- Payroll fixtures cannot enter payment, declaration, or certified-close workflows.
- Production runtime rejects sandbox and deferred adapters.
- Cross-tenant outbox claims and reconciliation are impossible.

Rollback strategy

All database changes are additive.

If runtime rollout fails:

1. Deploy the previous application version.
2. Leave new outbox enum values and records intact.
3. Stop the fiscalization worker.
4. Resume existing inline fiscal-document creation if necessary.
5. Reconcile queued records idempotently after the correction.

Existing fiscal-document uniqueness constraints prevent duplicate creation. Deferred records must not be deleted during rollback.

Acceptance criteria

The architecture is complete when:

- Pull-request CI requires no country-pack expert approval.
- `verify:repo` passes with development country-pack readiness.
- `verify:release` remains blocked until production evidence is valid.
- Zero feature modules import country-pack implementation files directly.
- Every eligible POS sale creates exactly one durable fiscalization request.
- POS completion succeeds during country-pack unavailability.
- No simulated value reaches a production payment, declaration, or certified close.
- Production cannot select a non-production regulatory adapter.
- Deferred obligations reconcile deterministically without rewriting history.
- Existing tenant, ledger, payment, inventory, audit, and authorization controls continue passing.

Final decision

Implement the **Regulatory Isolation Boundary**.

Start with the CI split because it delivers immediate relief with minimal risk. Then introduce the port and transactional-outbox fiscalization path. This leaves country-pack approval out of normal development while keeping every production regulatory outcome explicit, traceable, and fail-closed.